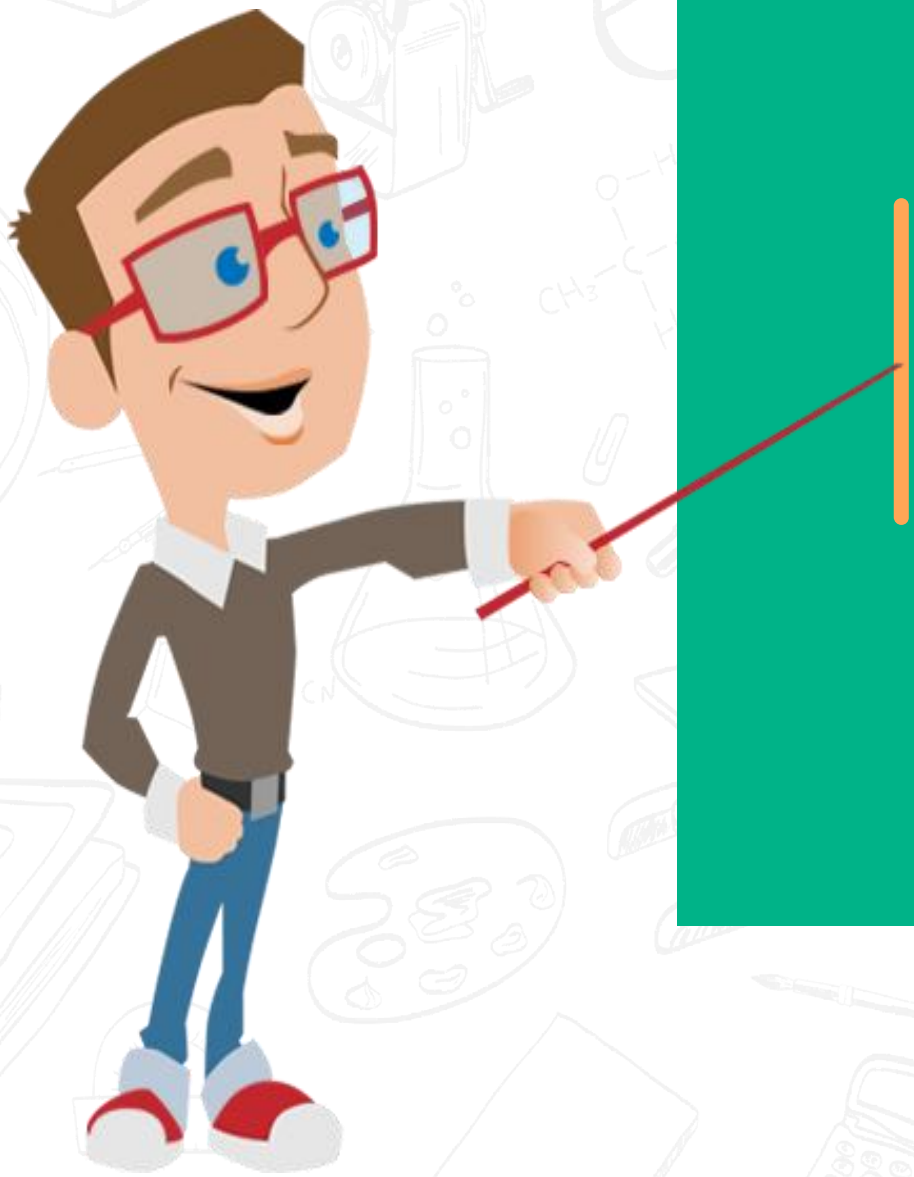


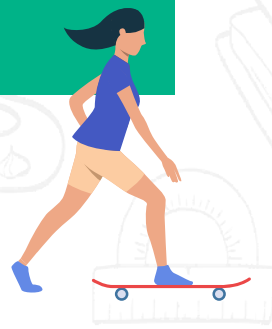
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 13-1

1 URDF Overview

2 URDF Syntax

3 URDF Practice

4 ~

Objectives

1、了解 URDF 的概念，以及 ROS 中搭建仿真环境的方法

2、掌握 URDF 基本语法，能够编写机器人模型的 URDF 文件



Pre-assessment

1、是否了解 URDF 的作用?

2、是否了解 Xacro?

3、是否自己建立过机器人仿真模型?



1

URDF Overview

1.1 URDF File

1.1.1 Overview

- 场景1：机器人价格高昂，学习ROS要购买一台机器人吗?
- 场景2：机器人交互的外界环境多样复杂，如何实现多样化环境中的性能测试?
- 场景3：进行算法测试时，直接将未经验证的程序部署到实体机器人运行，安全吗?

解决方案

- 机器人系统仿真：通过计算机对实体机器人系统进行模拟的技术
- 在 ROS 中，仿真实现涉及的内容有三部分：
  - 机器人建模(URDF)
  - 创建仿真环境(Gazebo)
  - 数据可视化(Rviz)

1.1 URDF File

1.1.1 Overview

1. 仿真优势：

- **降低成本**：仿真可以降低成本，减小风险
- **灵活高效**：搭建的环境更为灵活多样，可以提高测试效率以及测试覆盖率
- **高安全性**：仿真环境下，无需考虑耗损问题

2. 仿真劣势：

- **失真**：仿真所用的物理引擎无法完全模拟真实的物理世界
- **失准**：仿真器构建的是关节驱动器、传感器与信号通信的**绝对理想情况**，还无法实现模拟实际硬件缺陷或者一些临界状态等情形

3. 仿真三大件：(1) URDF (2) Rviz (3) Gazebo

1.1 URDF File

1.1.1 Overview

(1) URDF：创建机器人模型　　https://wiki.ros.org/urdf

- URDF 是 Unified Robot Description Format 的首字母缩写，直译为统一(标准化)机器人描述格式
- URDF 是一种 XML 文件，以 XML 的方式描述机器人的结构，比如底盘、摄像头、激光雷达、机械臂以及不同关节的自由度
- URDF 可以被 C++ 内置的解释器转换成可视化的机器人模型，是 ROS 中实现机器人仿真的重要组件
- URDF 需要结合 Gazebo 或 Rviz 使用，在 Gazebo 或 Rviz 中可以将 URDF 文件解析为图形化的机器人模型，一般的使用组合为：
  - 对于非仿真环境，使用 URDF 结合 Rviz 直接显示感知的真实环境信息
  - 对于仿真环境，使用 URDF 结合 Gazebo 搭建仿真环境，并结合 Rviz 显示感知的虚拟环境信息

1.1.1 Overview

(2) Rviz：图形化显示机器人模型、传感器感知到的数据信息

- Rviz 是 ROS Visualization Tool 的首字母缩写，直译为 ROS 的三维可视化工具
- Rviz 主要用于以三维方式显示 ROS 消息，以及将数据进行可视化表达
- 举例：显示机器人模型、激光测距仪传感器检测到的障碍物距离、Kinect 三维距离传感器的点云数据、从相机获取的图像值等

1.1 URDF File

1.1.1 Overview

(3) Gazebo：搭建仿真环境

- Gazebo是一款 3D 动态模拟器，用于显示机器人模型并创建仿真环境
- Gazebo 能够在复杂的室内和室外环境中准确有效地模拟机器人
- 与游戏引擎提供高保真度的视觉模拟类似，Gazebo 提供高保真度的物理模拟以及各类传感器模型，Gazebo 为用户和程序提供友好的交互方式
- 安装：

```
$ sudo apt install gazebo11
```

```
$ sudo apt install libgazebo11-dev
```

- 运行：

```
$ rosrun gazebo_ros gazebo
```

```
$ gazebo
```



2

URDF Syntax

2.1 URDF Format

2.1.1 Overview：https://wiki.ros.org/urdf/XML

- URDF 文件是一个标准的 XML 文件，在 ROS 中预定义了一系列的标签用于描述机器人模型
- URDF 的标签可以简化为两部分：连杆(link 标签) 与 关节(joint 标签)

  (1) robot 根标签

  (2) link 连杆标签

  (3) joint 关节标签

  (4) gazebo 标签

2.1.2 Link

robot 根标签：

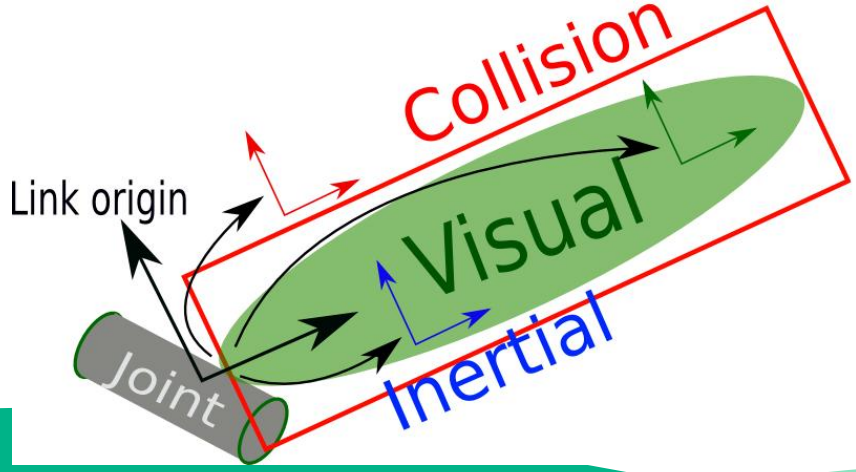
- URDF 中为了保证 XML 语法的完整性，使用了 robot 标签作为根标签，所有的 link 和 joint 以及其他标签都必须包含在 robot 标签内，在该标签内可以通过 name 属性设置机器人模型的名称
- 属性
  - name=“机器人模型的名称”
- 子标签
  - 其他标签都是 robot 的子级标签

2.1 URDF Format

2.1.2 Link

link 标签：

- URDF 中的 link 标签用于描述机器人刚体部件的外观和物理属性。比如：机器人底座、轮子、激光雷达、摄像头等
- 每一个部件都对应一个 link。在 link 标签内，可以设计该部件的形状、尺寸、颜色、惯性矩阵、碰撞参数等一系列属性

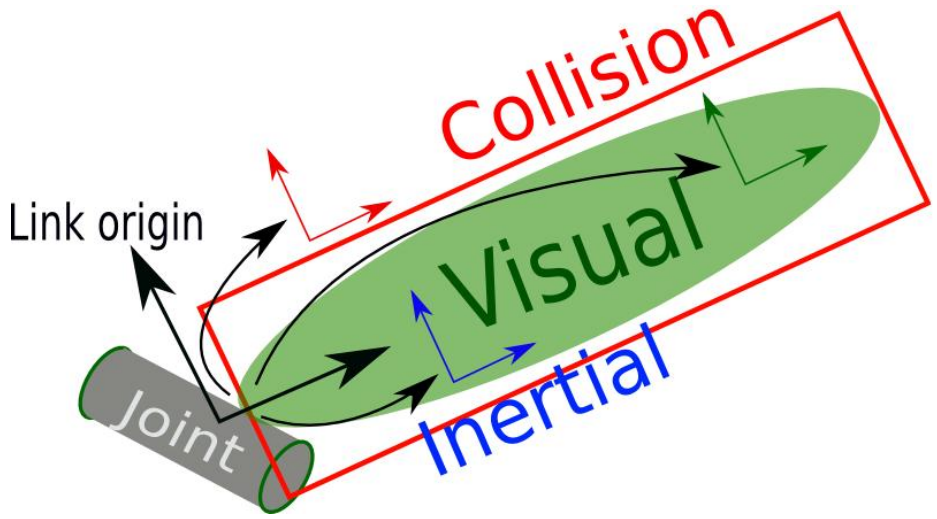


2.1 URDF Format

2.1.2 Link

link 标签：

- 属性
  - name=“连杆名称”
- 子标签
  - visual：描述零件外观(对应的数据是可视的)
  - collision：描述连杆的碰撞属性
  - inertial：描述连杆的惯性矩阵



2.1.2 Link

- visual：描述零件外观
  - geometry：设置零件的形状
  - origin：设置零件的偏移量与倾斜角度
  - material：设置零件的材料属性

2.1 URDF Format

2.1.2 Link

- geometry：设置零件的形状
  - ✓ box：盒状
    - 属性：size=“长 宽 高”
  - ✓ cylinder：圆柱
    - 属性：radius=“半径” length=“高度”
  - ✓ sphere：球体
    - 属性：radius=“半径”
  - ✓ mesh：网格模型
    - 属性：filename=“模型路径” (格式：package://<packagename>/<path>/file)

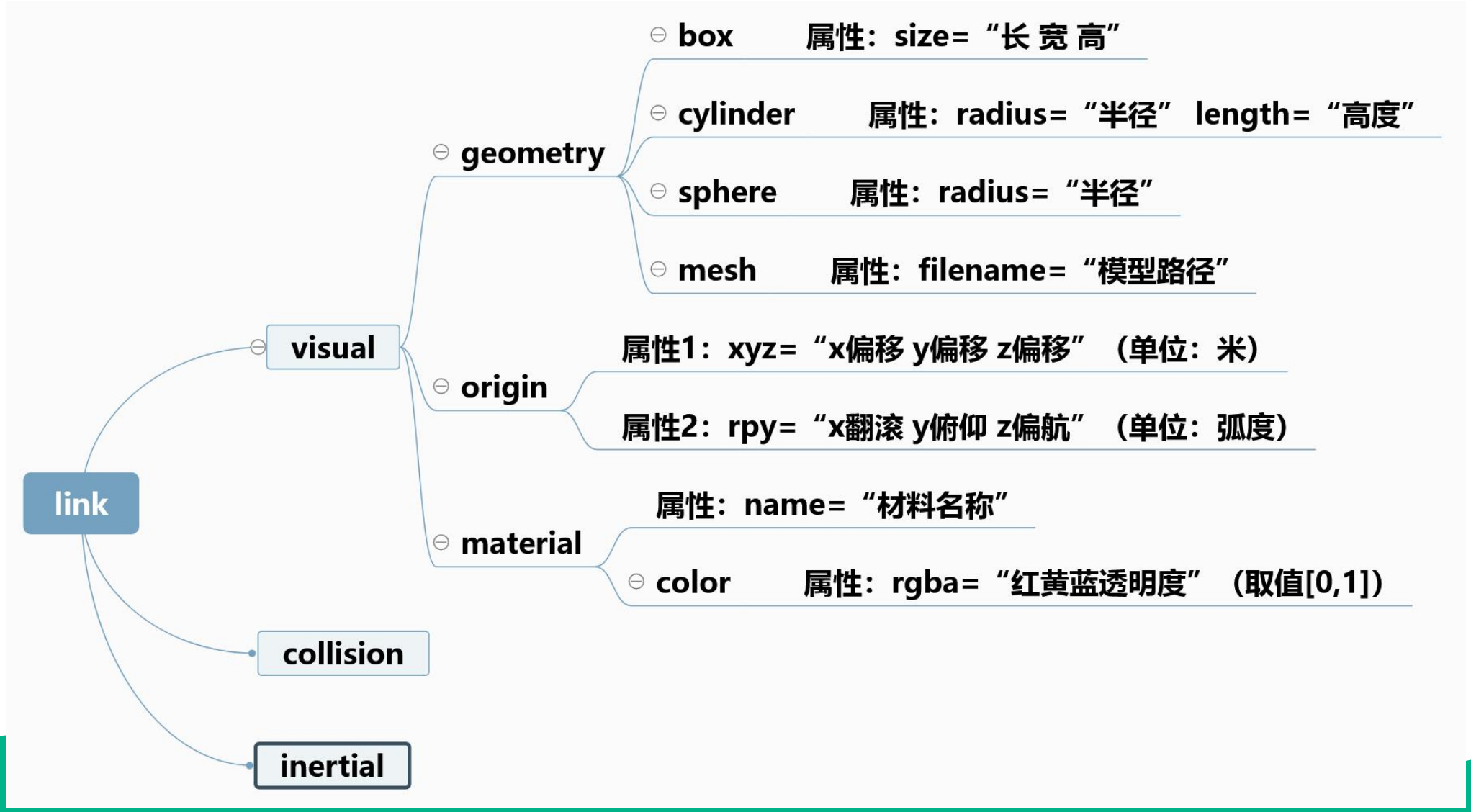
2.1 URDF Format

2.1.2 Link

- origin：设置零件的偏移量与倾斜角度，以零件自身的几何中心为基准
  - 属性1：xyz=“x偏移 y偏移 z偏移” （单位：米）
  - 属性2：rpy=“x翻滚 y俯仰 z偏航” （单位：弧度）
- material：设置零件的材料属性
  - 属性：name=“材料名称”
  - 标签：color
    - 属性：rgba=“红黄蓝透明度” （取值[0,1]）

2.1 URDF Format

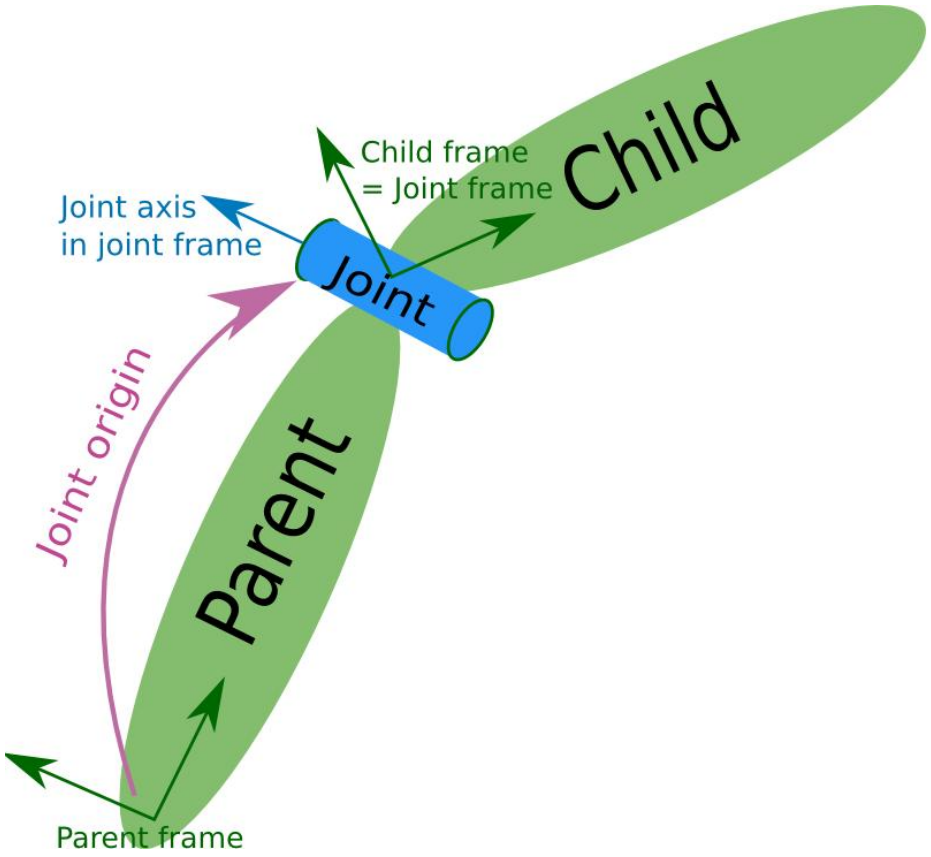
2.1.2 Link



2.1 URDF Format

2.1.3 Joint

- URDF 中的 joint 标签用于描述机器人关节的运动学和动力学属性
- 可以指定关节运动的安全极限，机器人的两个部件(分别称之为 parent link 与 child link)以"关节"的形式相连接
- 不同的关节有不同的运动形式: 旋转、滑动、固定、旋转速度、旋转角度限制等
- 举例：安装在底座上的轮子可以 360 度旋转，摄像头完全固定在底座上等应用场景



2.1.3 Joint

joint标签：

- 属性
  - name=“关节命名”
  - type=“关节类型”，关节类型可取如下值：
    - ✓ continuous: 旋转关节，可以绕单轴无限旋转
    - ✓ revolute: 旋转关节，类似于 continues，但是有旋转角度限制
    - ✓ prismatic: 滑动关节，沿某一轴线移动的关节，有位置极限
    - ✓ planer: 平面关节，允许在平面正交方向上平移或旋转
    - ✓ floating: 浮动关节，允许进行平移、旋转运动
    - ✓ fixed: 固定关节，不允许运动的特殊关节

2.1 URDF Format

2.1.3 Joint

joint标签：

- 子标签
  - parent(必需的)，parent link 是一个强制属性：
    - ✓ link=“父级连杆的名称”
  - child(必需的)，child link 是一个强制属性：
    - ✓ link=“子级连杆的名称”
  - origin　此处的 origin 以 parent link 的几何中心为基准
    - ✓ 属性: xyz=“xyz轴线上的偏移量” rpy=“xyz轴线上的偏移角度”
  - axis
    - ✓ 属性: xyz=“设置围绕哪个关节轴运动”

2.1.4 案例

案例：在 Rviz 中显示建立的机器人模型步骤

1. 准备：新建功能包，导入依赖
2. 核心：编写 URDF 文件
3. 核心：在 launch 文件集成 URDF 与 Rviz
4. 在 Rviz 中显示机器人模型
5. 启动 launch 文件

2.1.4 案例

1. 准备：新建功能包，导入依赖

- 创建一个新的功能包
  - 功能包名称：rcdp
  - 添加依赖项：urdf xacro gazebo_ros gazebo_ros_control gazebo_plugins
- 在当前功能包下，创建以下目录：
  - urdf: 存放 URDF 文件的目录
    - ✓ urdf目录下，新建 URDF 和 Xacro 文件夹
  - meshes：存放机器人模型网格文件
  - config: 存放配置文件
  - launch: 存放 launch 启动文件

2.1 URDF Format

2.1.4 案例

ROS中URDF格式和使用：
https://blog.csdn.net/shenyan0712/article/details/89919959

2. 核心：根据 URDF 规则编写 URDF 文件

3. Launch文件：在 launch 文件集成 URDF 与 Rviz

```
<launch>

  <!-- 设置参数 -->
  <param name="robot_description" textfile="$(find rs_urdf)/urdf/urdf/myrobot.urdf" />

  <!-- 启动 rviz -->
  <node pkg="rviz" type="rviz" name="rviz" />
  <!-- <node pkg="rviz" type="rviz" name="rviz" args="-d $(find rs_urdf)/config/myrobot.rviz"/> -->


</launch>
```

在launch文件中导入URDF文件的方法：使用参数名：robot_description，将 URDF 文件的路径设置到参数服务器

2.1.4 案例

4. Rviz 参数设置

- 重复启动 launch 文件时，Rviz 之前的组件配置信息不会自动保存
- 为了方便使用，可以使用如下方式优化：
  - 第一步，将当前配置保存进 config 目录
  - 第二步，launch文件中 Rviz 的启动配置添加参数：args，值设置为 -d 配置文件路径

```
<launch>

  <!-- 设置参数 -->
  <param name="robot_description" textfile="$(find rs_urdf)/urdf/urdf/myrobot.urdf" />

  <!-- 启动 rviz -->
  <!-- <node pkg="rviz" type="rviz" name="rviz" /> -->
  <node pkg="rviz" type="rviz" name="rviz" args="-d $(find rs_urdf)/config/myrobot.rviz"/>


</launch>
```

2.1.4 案例

5. 启动：在 Rviz 中显示机器人模型

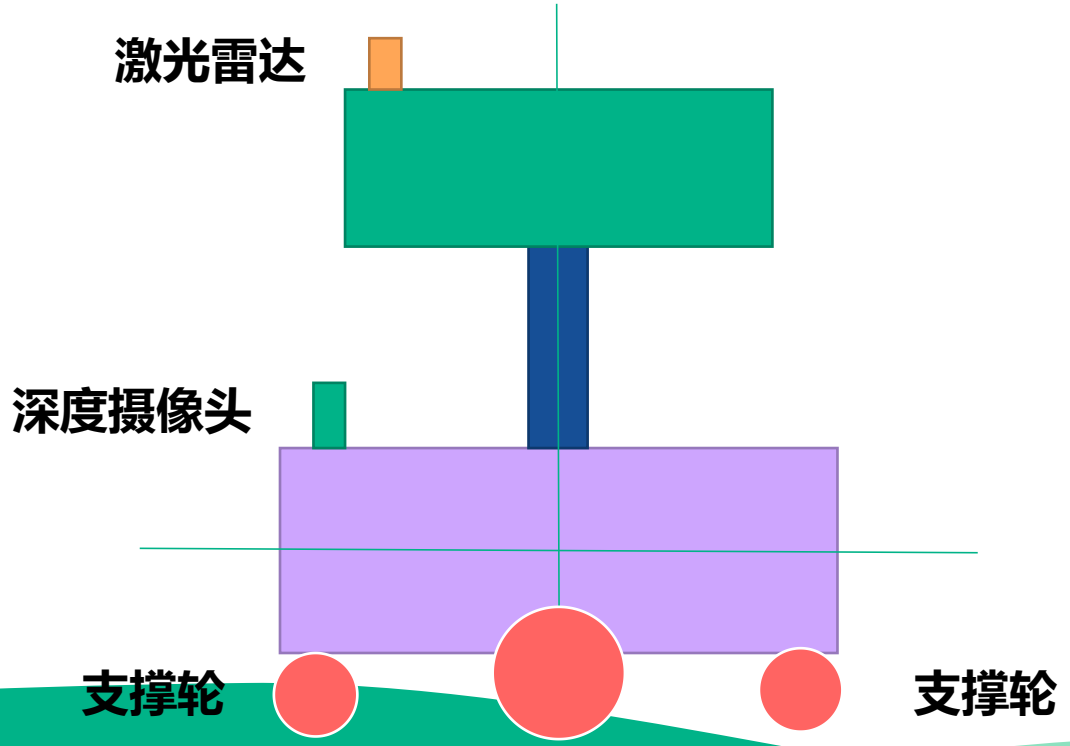
ctrl+shift+b 编译

roslaunch 功能包 launch文件名

2.1 URDF Format

2.1.5 实践

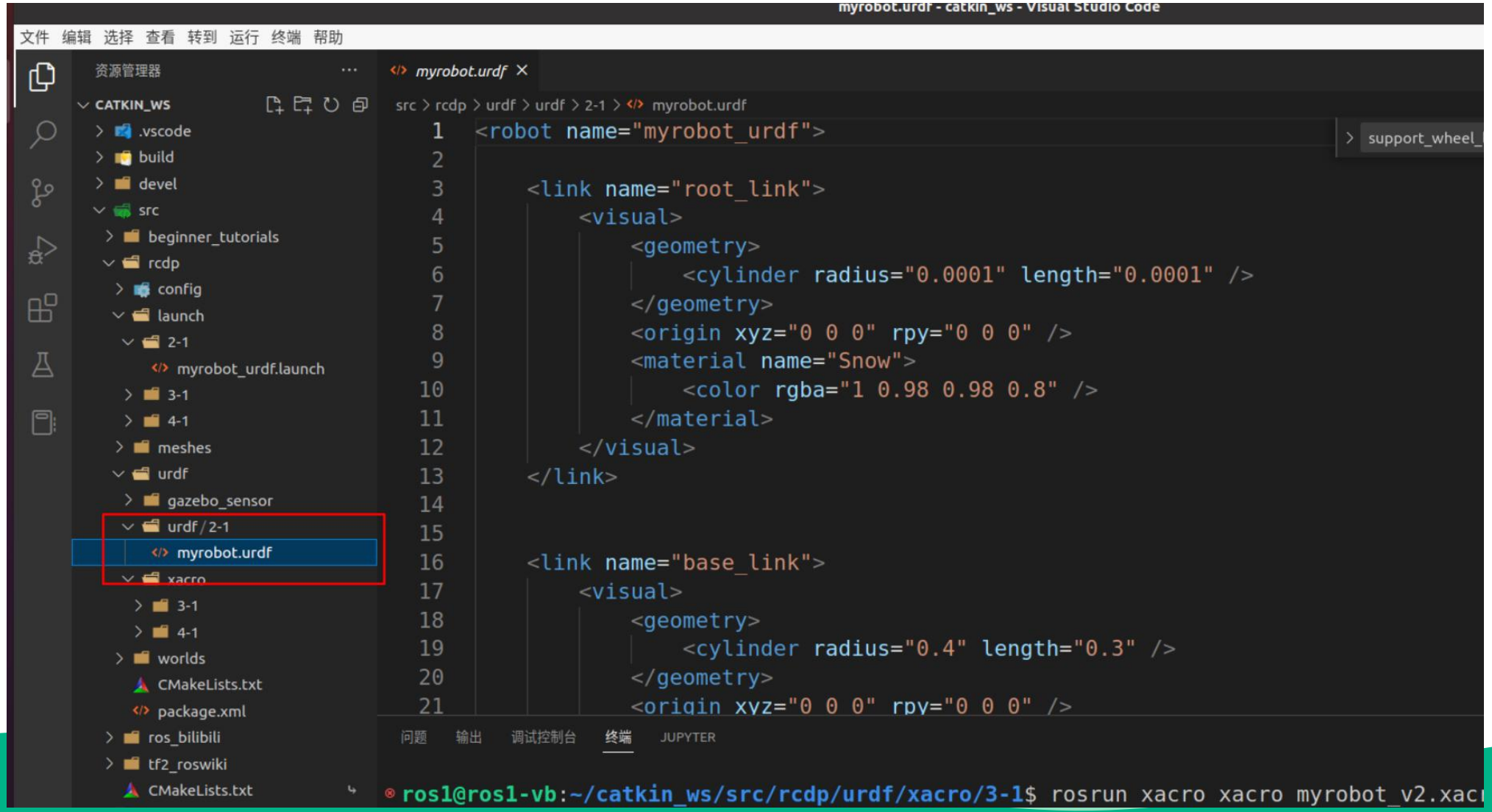
实践内容：创建差速机器人模型，该模型为双层结构，中间由圆柱形连杆连接；机器人底盘为圆柱体，在圆柱体的前面放置一个深度摄像头，摄像头可沿着Z轴 -180 到 180 旋转；机器人二层为圆柱体，在圆柱体前面放置一个激光雷达，雷达可以沿着 Z 轴 360 度旋转



2.1 URDF Format

2.1.5 实践

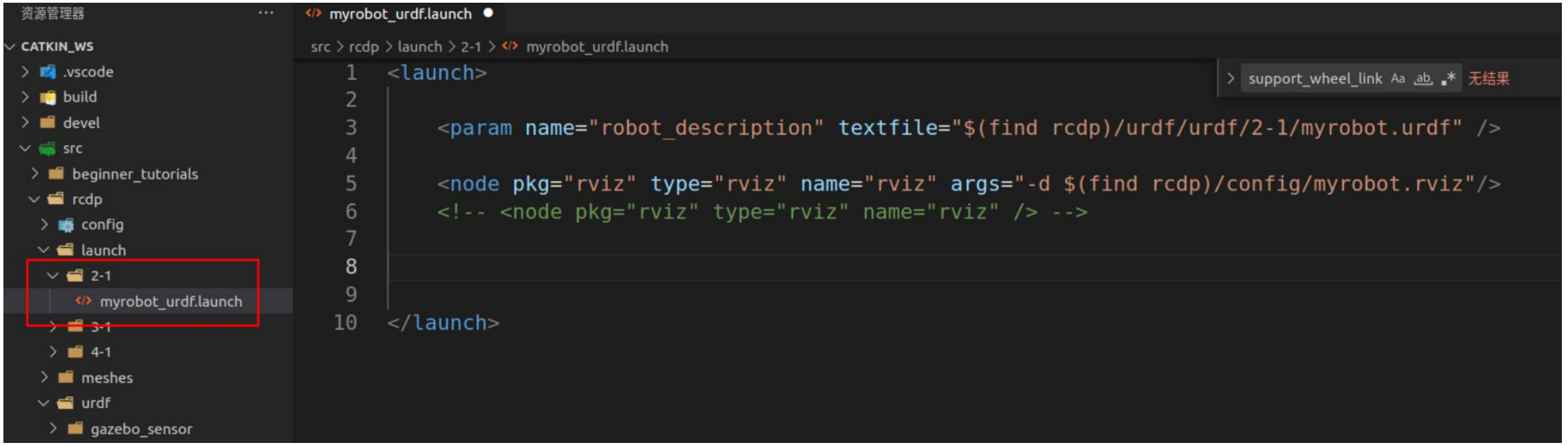
URDF文件:



2.1 URDF Format

2.1.5 实践

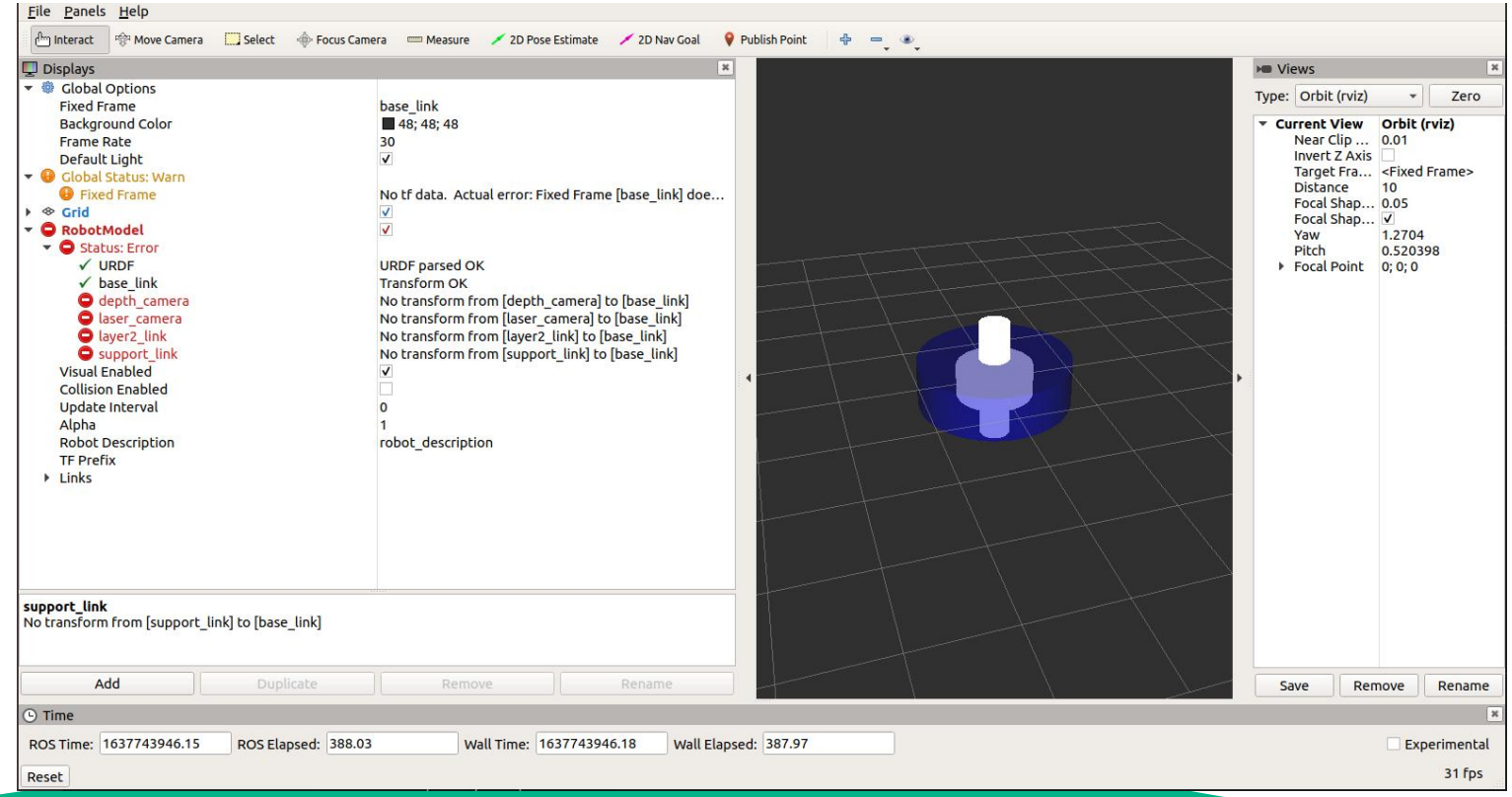
launch文件：



2.1 URDF Format

2.1.5 实践

- 运行结果：



原因：

rviz 中显示 URDF时，必须发布不同零件之间的 **坐标系变换** 关系

解决方案：

ROS中提供了关于机器人模型显示的坐标系关系节点，分别为：

joint_state_publisher(功能包同名)

robot_state_publisher(功能包同名)

在launch文件中包含这两个节点

2.1 URDF Format

2.1.5 实践

- 2个重要功能包：

joint_state_publisher：

- joint_state_publisher是一个ROS功能包，它的主要功能是发布**关节状态信息**，包括**关节角度、速度、加速度**等。关节状态信息是从机器人硬件接收，并转换为ROS中的JointState消息
- 通过读取参数服务器上的/robot_description来获得机器人的所有非固定关节
- 节点会将关节状态信息发布到/joint_state话题，这个话题可以被其他ROS节点如运动控制节点、仿真节点等订阅。
- 这个节点发布的频率一般是10Hz，如果发布的话题频率超过10Hz，关节角度可能会出现来回跳动的情况

2.1.5 实践

- 2个重要功能包：

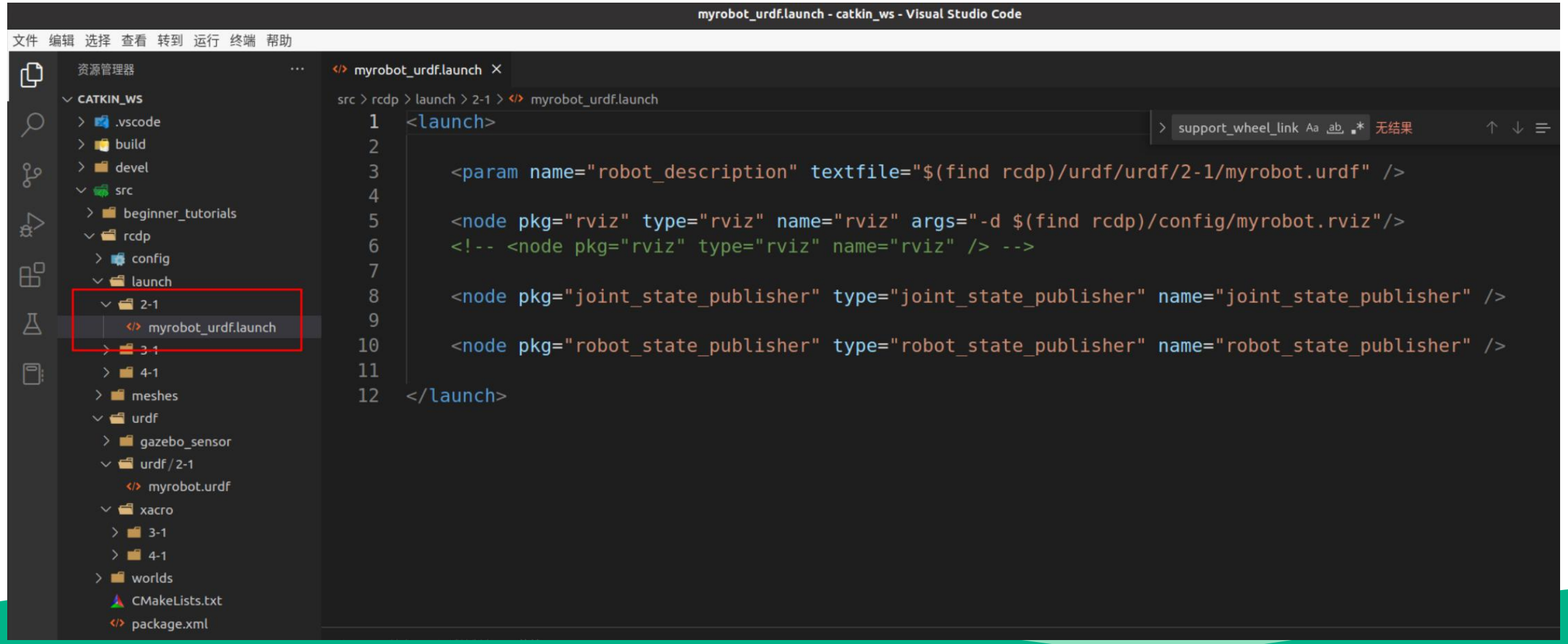
robot_state_publisher：

- robot_state_publisher是一个ROS软件包，它的主要功能是发布机器人的完整状态信息。除了关节状态信息之外，robot_state_publisher还发布了机器人的TF变换信息，包括机器人坐标系和关节坐标系之间的变换关系
- 在实际应用中，robot_state_publisher节点通常在joint_state_publisher节点的工作基础之上工作。它根据创建的描述机器人装配的URDF模型，编写一个节点来对机器人的运动进行仿真，并发布JointState和平移消息。然后，robot_state_publisher节点会将整个机器人的状态消息发布到/tf2话题上，以供系统中其他使用tf的组件获取和使用

2.1 URDF Format

2.1.5 实践

launch文件：



2.1 URDF Format

2.1.5 实践

- 运行结果：



原因：

URDF文件中包含注释

解决方案：

删除URDF文件中的注释

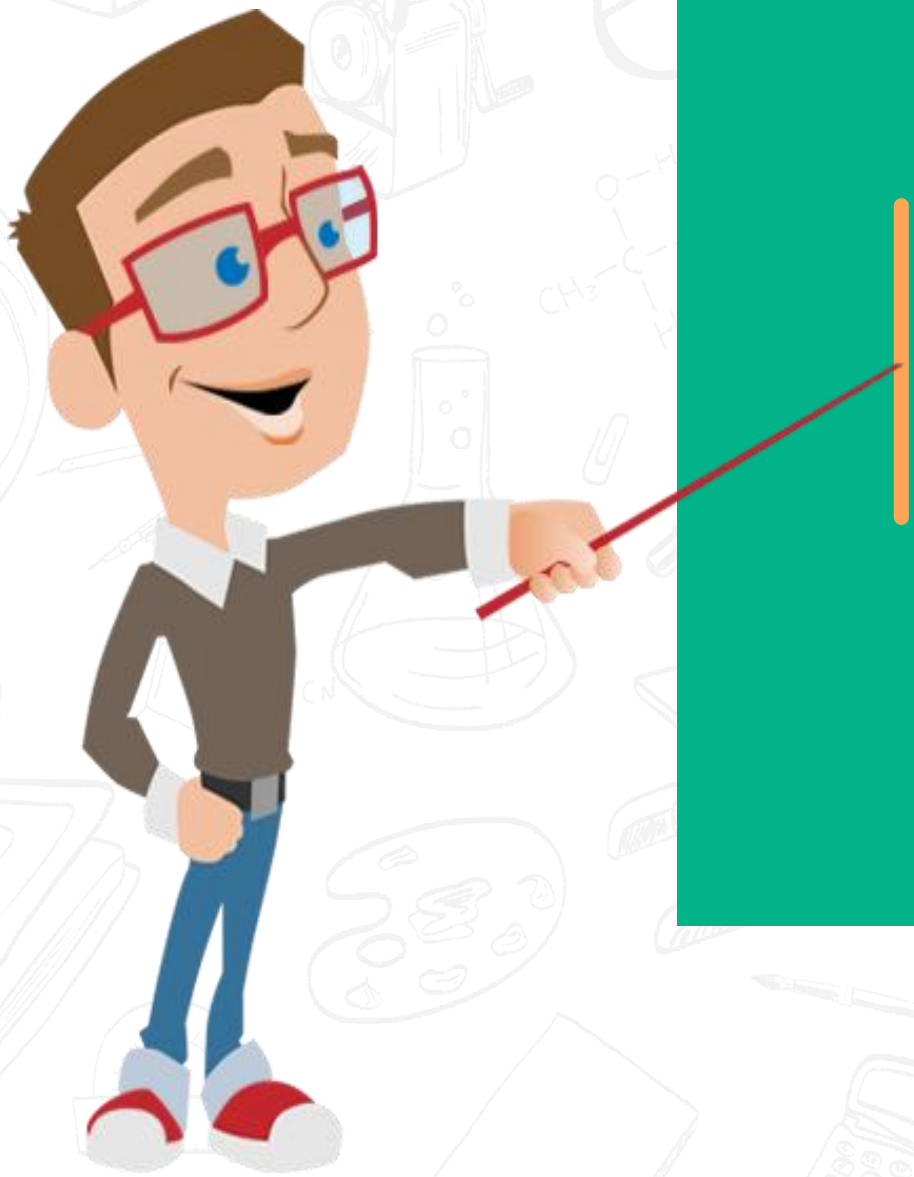
Summary

1. URDF Overview
    a) 作用
    b) 优劣势
    c) 定义和使用方法
    d) Rviz 和 Gazebo简介
2. URDF Syntax
    a) robot 标签
    b) link 标签
    c) joint 标签
    d) 案例演示

Q&A

今日事今日毕

Question?



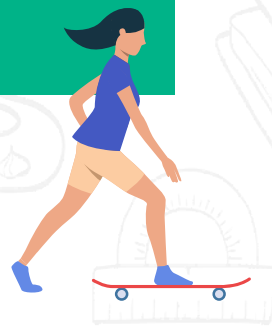
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 13-2

1. Xacro Overview
2. Xacro Syntax
3. Xacro Practice
4. Arbotix Controller

Objectives

1、了解 Xacro 的概念，以及 ROS 中搭建仿真环境的方法

2、掌握 Xacro 基本语法，能够编写机器人模型的 Xacro 文件

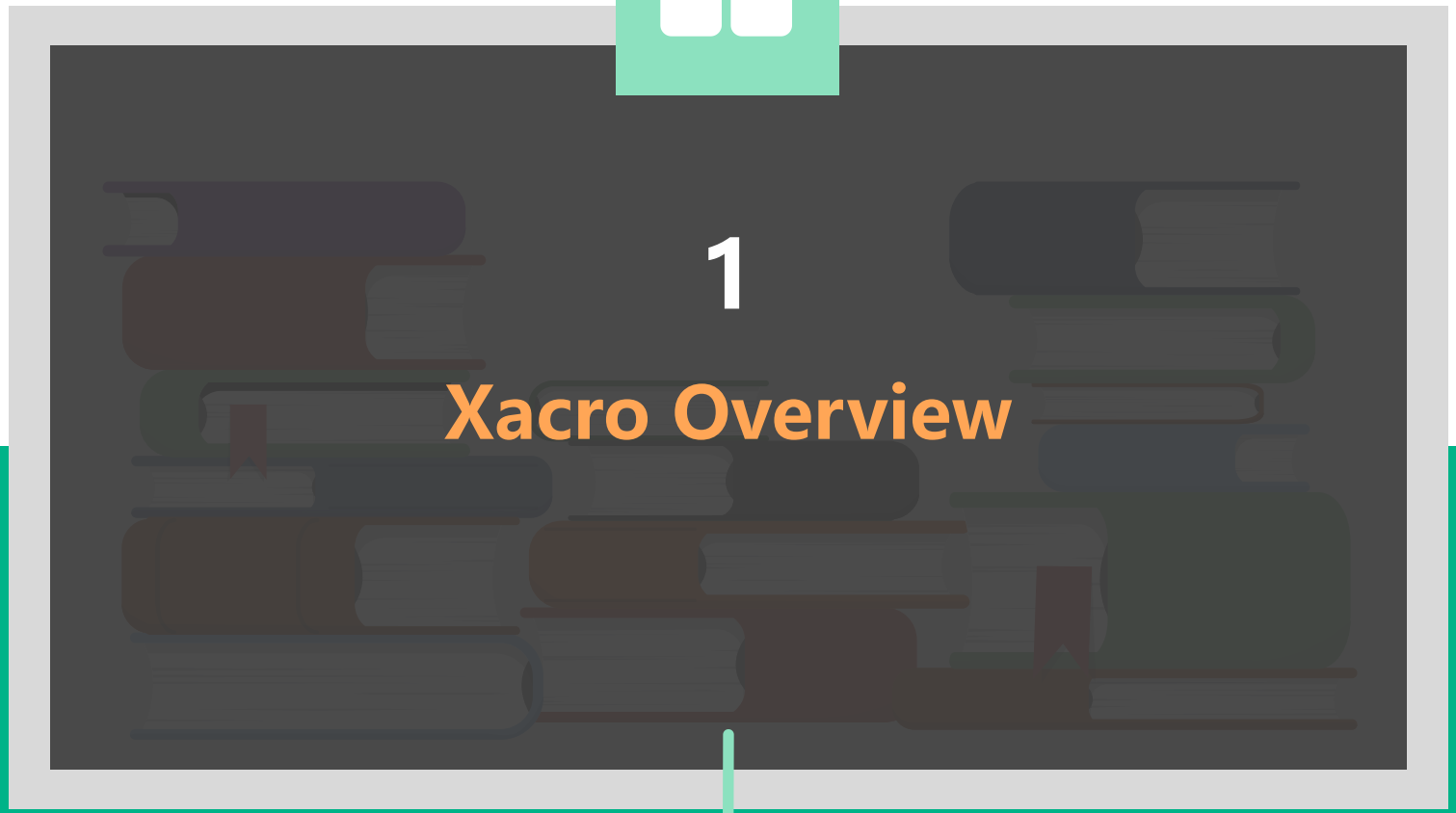
3、了解 Arbotix 控制器的配置和使用方法



Pre-assessment

1、是否了解 Xacro 的作用?

2、是否编写过 Xacro 文件?



1

Xacro Overview

1.1 URDF Review

1.1.1 Overview

(1) URDF：创建机器人模型　　https://wiki.ros.org/urdf

- URDF是一种 XML 文件，以 XML 的方式描述机器人的结构，比如底盘、摄像头、激光雷达、机械臂以及不同关节的自由度；URDF 需要结合 Gazebo 或 Rviz 使用，在 Gazebo 或 Rviz 中可以将 URDF 文件解析为图形化的机器人模型

(2) Rviz：图形化的显示机器人各种传感器感知到的环境信息

- Rviz 是 ROS 的三维可视化工具，主要用于以三维方式显示 ROS 消息，将数据进行可视化表达
- 举例：显示机器人模型、显示表达激光测距仪传感器检测到的障碍物距离、Kinect 三维距离传感器的点云数据显示、从相机获取的图像值等

(3) Gazebo：搭建仿真环境

- Gazebo 是一款 3D 动态模拟器，用于显示机器人模型并创建仿真环境
- Gazebo 能够在复杂的室内和室外环境中准确有效地模拟机器人
- Gazebo 提供高保真度的物理模拟，其提供一整套传感器模型，以及对用户和程序非常友好的交互方式

1.2 URDF Syntax

1.2.1 Overview：https://wiki.ros.org/urdf/XML

- URDF 文件是一个标准的 XML 文件，在 ROS 中预定义了一系列的标签用于描述机器人模型
- URDF的标签可以简化为两部分：连杆(link标签) 与 关节(joint标签)

  (1) robot 根标签

  (2) link 连杆标签

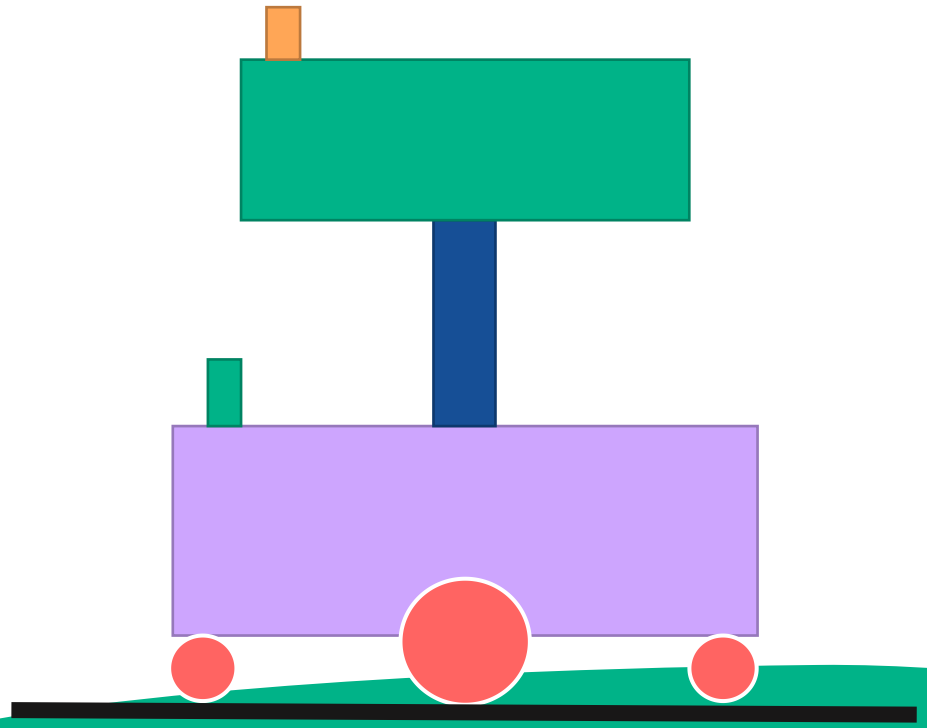
  (3) joint 关节标签

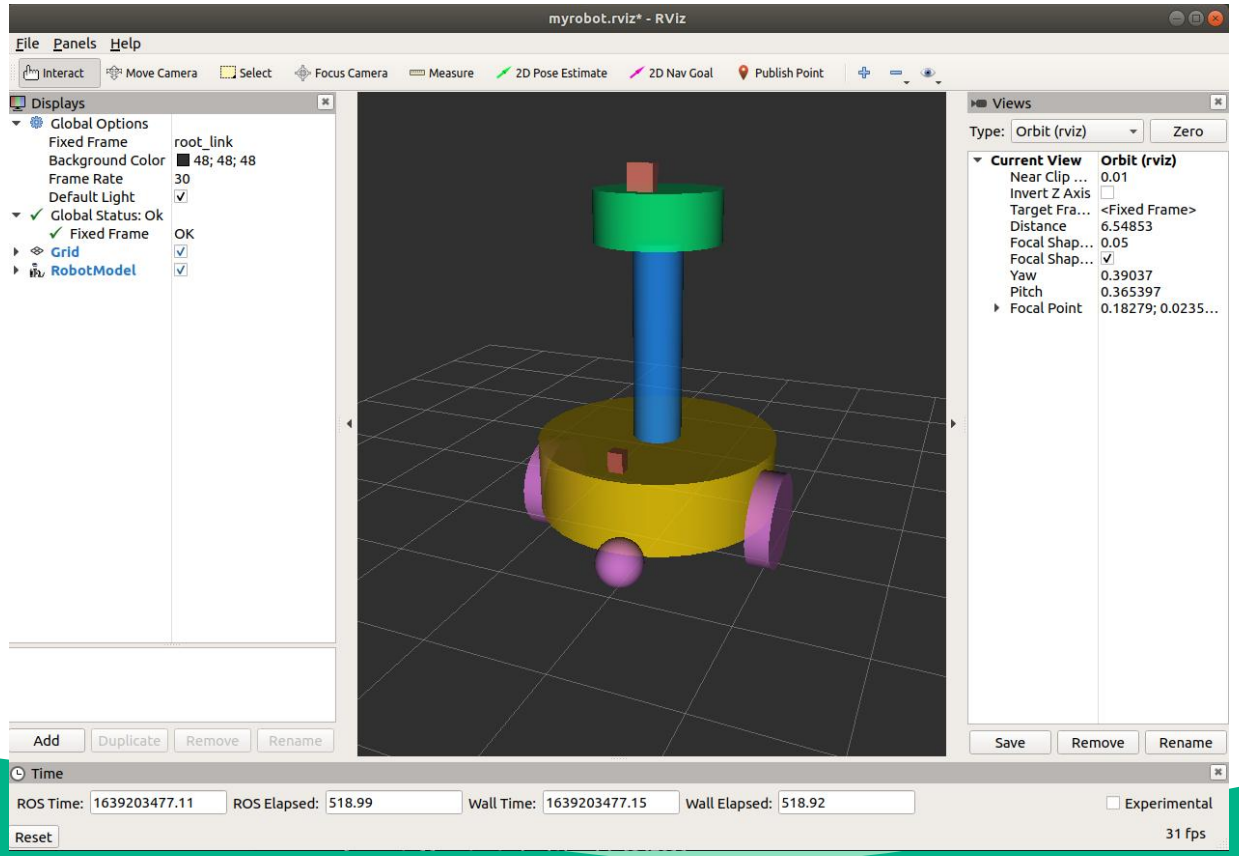
  (4) gazebo 标签

1.3 URDF Demo

1.3.1 Overview：https://wiki.ros.org/urdf/XML

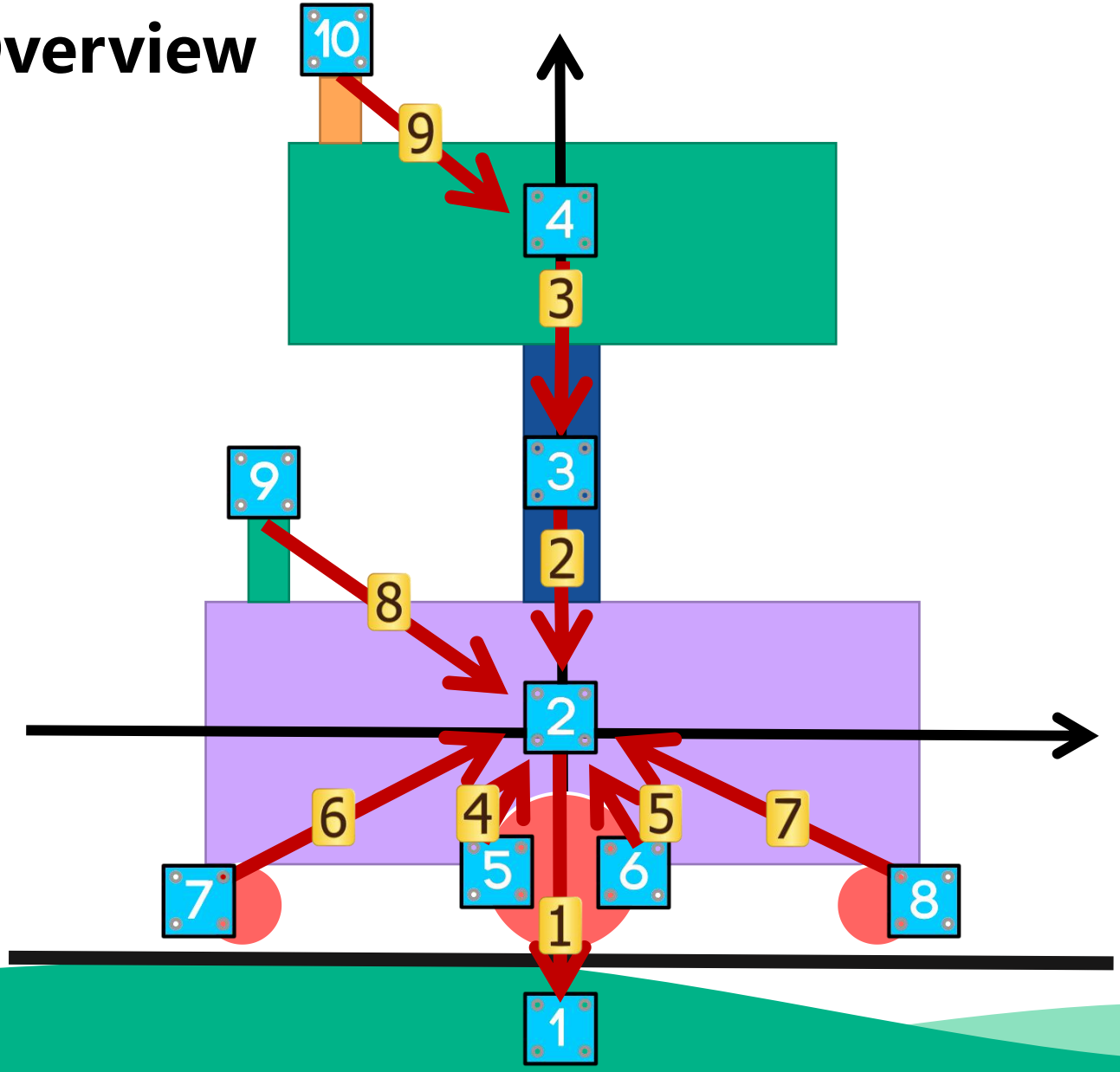
案例需求：创建差速机器人模型，该模型为双层结构，中间由圆柱形连杆连接；机器人底盘为圆柱体，在圆柱体的前面放置一个深度摄像头，摄像头可沿着Z轴 -180 到 180 旋转；机器人二层为圆柱体，在圆柱体前面放置一个激光雷达，雷达可以沿着 Z 轴 360 度旋转





1.3 URDF Demo

1.3.1 Overview



- 蓝色数字：link
- 黄色数字：joint
- 使用元素：
  - 球
  - 圆柱
  - 长方体

1.3 URDF Demo

1.3.2 URDF File

- 代码分析：link标签

```
<robot name="myrobot_urdf">

    <link name="root_link">
        <visual>
            <geometry>
                <cylinder radius="0.0001" length="0.0001" />
            </geometry>
            <origin xyz="0 0 0" rpy="0 0 0" />
            <material name="Snow">
                <color rgba="1 0.98 0.98 0.8" />
            </material>
        </visual>
    </link>


    <link name="base_link">
        <visual>
            <geometry>
                <cylinder radius="0.4" length="0.3" />
            </geometry>
            <origin xyz="0 0 0" rpy="0 0 0" />
            <material name="Gold">
                <color rgba="1 0.843 0 0.8" />
            </material>
        </visual>
    </link>
```

```
<link name="front_support_wheel_link">
    <visual>
        <geometry>
            <sphere radius="0.05" />
        </geometry>
        <origin xyz="0 0 0" rpy="0 0 0" />
        <material name="Voilet">
            <color rgba="0.933 0.51 0.933 0.8" />
        </material>
    </visual>
</link>


<link name="back_support_wheel_link">
    <visual>
        <geometry>
            <sphere radius="0.05" />
        </geometry>
        <origin xyz="0 0 0" rpy="0 0 0" />
        <material name="Voilet">
            <color rgba="0.933 0.51 0.933 0.8" />
        </material>
    </visual>
</link>
```

1.3 URDF Demo

1.3.2 URDF File

- 代码分析：link标签

```
<link name="left_wheel_link">
    <visual>
        <geometry>
            <cylinder radius="0.1" length="0.08" />
        </geometry>
        <origin xyz="0 0 0" rpy="1.57 0 0" />
        <material name="Voilet">
            <color rgba="0.933 0.51 0.933 0.8" />
        </material>
    </visual>
</link>


<link name="right_wheel_link">
    <visual>
        <geometry>
            <cylinder radius="0.1" length="0.08" />
        </geometry>
        <origin xyz="0 0 0" rpy="1.57 0 0" />
        <material name="Voilet">
            <color rgba="0.933 0.51 0.933 0.8" />
        </material>
    </visual>
</link>
```

```
<link name="support_link">
    <visual>
        <geometry>
            <cylinder radius="0.1" length="0.8" />
        </geometry>
        <origin xyz="0 0 0" rpy="0 0 0" />
        <material name="DodgerBlue">
            <color rgba="0.118 0.565 1 0.8" />
        </material>
    </visual>
</link>


<link name="platform_link">
    <visual>
        <geometry>
            <cylinder radius="0.25" length="0.15" />
        </geometry>
        <origin xyz="0 0 0" rpy="0 0 0" />
        <material name="SpringGreen">
            <color rgba="0 1 0.498 0.8" />
        </material>
    </visual>
</link>
```

1.3 URDF Demo

1.3.2 URDF File

- 代码分析：link标签

```
<link name="laser_scan_link">
    <visual>
        <geometry>
            <box size="0.05 0.06 0.08" />
        </geometry>
        <origin xyz="0 0 0" rpy="0 0 0" />
        <material name="IndianRed">
            <color rgba="0.804 0.361 0.361 0.8" />
        </material>
    </visual>
</link>


<link name="depth_camera_link">
    <visual>
        <geometry>
            <box size="0.05 0.08 0.08" />
        </geometry>
        <origin xyz="0 0 0" rpy="0 0 0" />
        <material name="Salmon">
            <color rgba="0.981 0.502 0.447 0.8" />
        </material>
    </visual>
</link>
```

1.3 URDF Demo

1.3.2 URDF File

- 代码分析：join标签

```
<joint name="base_link2root_link" type="fixed">
    <parent link="root_link"/>
    <child link="base_link" />
    <origin xyz="0 0 0.25" rpy="0 0 0" />
    <axis xyz="0 0 0" />
</joint>


<joint name="front_support_wheel_link2base_link" type="continuous">
    <parent link="base_link"/>
    <child link="front_support_wheel_link" />
    <origin xyz="0.32 0 -0.2" rpy="0 0 0" />
    <axis xyz="1 1 1" />
</joint>


<joint name="back_support_wheel_link2base_link" type="continuous">
    <parent link="base_link"/>
    <child link="back_support_wheel_link" />
    <origin xyz="-0.32 0 -0.2" rpy="0 0 0" />
    <axis xyz="1 1 1" />
</joint>
```

```
<joint name="base_l_wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child link="left_wheel_link" />
    <origin xyz="0 0.4 -0.15" rpy="0 0 0" />
    <axis xyz="0 1 0" />
</joint>


<joint name="base_r_wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child link="right_wheel_link" />
    <origin xyz="0 -0.4 -0.15" rpy="0 0 0" />
    <axis xyz="0 1 0" />
</joint>


<joint name="support_link2base_link" type="fixed">
    <parent link="base_link"/>
    <child link="support_link" />
    <origin xyz="0 0 0.55" rpy="0 0 0" />
    <axis xyz="0 0 0" />
</joint>
```

1.3 URDF Demo

1.3.2 URDF File

- 代码分析：join标签

```
    <joint name="platform_link2support_link" type="fixed">
        <parent link="support_link"/>
        <child link="platform_link" />
        <origin xyz="0 0 0.475" rpy="0 0 0" />
        <axis xyz="0 0 0" />
    </joint>


    <joint name="depth_camera_link2base_link" type="revolute">
        <parent link="base_link"/>
        <child link="depth_camera_link" />
        <origin xyz="0.32 0 0.19" rpy="0 0 0" />
        <limit lower="-1.57" upper="1.57" effort="10" velocity="3"/>
        <axis xyz="0 0 1" />
    </joint>


    <joint name="laser_scan_link2platform_link" type="continuous">
        <parent link="platform_link"/>
        <child link="laser_scan_link" />
        <origin xyz="0.2 0 0.115" rpy="0 0 0" />
        <axis xyz="0 0 1" />
    </joint>

</robot>
```

1.4 Xacro Overview

1.4.1 URDF 问题分析

(1) 问题1：在设计关节的位置时，需要按照一定的公式计算，公式是固定的。但是在 URDF 中依赖于手工计算，存在不便，容易计算失误，且当某些参数发生改变时，还需要重新计算

(2) 问题2：URDF 中的部分内容是高度重复的，驱动轮与支撑轮的设计实现，不同轮子只是部分参数不同，形状、颜色、翻转量都是一致的；在实际应用中，构建复杂的机器人模型时，更是易于出现高度重复的设计，按照一般的编程涉及到重复代码应该考虑封装

可以通过 变量 结合 函数 直接解决上述问题。在 ROS 中，已经给出了类似编程的优化方案，称之为：Xacro

1.4 Xacro Overview

1.4.2 Xacro File：http://wiki.ros.org/xacro

➢ 概念：

Xacro 是 XML Macros 的缩写，Xacro 是一种 XML 宏语言，是可编程的 XML

➢ 原理

Xacro 可以 声明变量，可以通过 数学运算求解，使用 流程控制控制执行顺序，还可以通过 类似函数的实现，封装固定的逻辑，将逻辑中需要的可变的数据以参数的方式展示，提高代码复用率以及程序的安全性

➢ 作用

较之于 URDF 实现，可以编写更安全、精简、易读性更强的机器人模型文件，且可以提高编写效率



2

Xacro Syntax

2.1 Xacro basic Syntax

2.1.1 属性与算数运算 https://www.cnblogs.com/caiyishuai/p/15044043.html

(1) Xacro 提供了可编程接口，类似于计算机语言，包括 变量声明调用、函数声明与调用等语法实现。在使用 xacro 生成 URDF 时，根标签 robot 中必须包含命名空间声明：xmlns:xacro="http://wiki.ros.org/xacro"

(2) 属性与算数运算：

用于封装 URDF 中的一些字段，比如: PI 值，小车的尺寸，轮子半径

- 属性定义：

```
<xacro:property name="xxxx" value="yyyy" />
```

2.1 Xacro basic Syntax

2.1.1 属性与算数运算

- 属性调用：

${属性名称}

- 算数运算：

${数学表达式}

2.1 Xacro basic Syntax

2.1.2 Xacro宏

(1) 宏

类似于函数实现，提高代码复用率，优化代码结构，提高安全性

(2) 宏定义(函数定义)：

```
<xacro:macro name="宏名称" params="参数列表(多参数之间使用空格分隔)">
    参数调用格式: ${参数名}
</xacro:macro>
```

(3) 宏调用(函数调用)：

```
<xacro:宏名称 参数1=xxx 参数2=xxx/>
```

2.1.3 文件包含

(1) 文件包含

机器人由多部件组成，不同部件可能封装为单独的 xacro 文件，最后再将不同的文件集成，组合为完整机器人，可以使用文件包含实现

(2) 语法结构：

```
<robot name="xxx" xmlns:xacro="http://wiki.ros.org/xacro">
    <xacro:include filename="my_base.xacro" />
    <xacro:include filename="my_camera.xacro" />
    <xacro:include filename="my_laser.xacro" />
    ....
</robot>
```



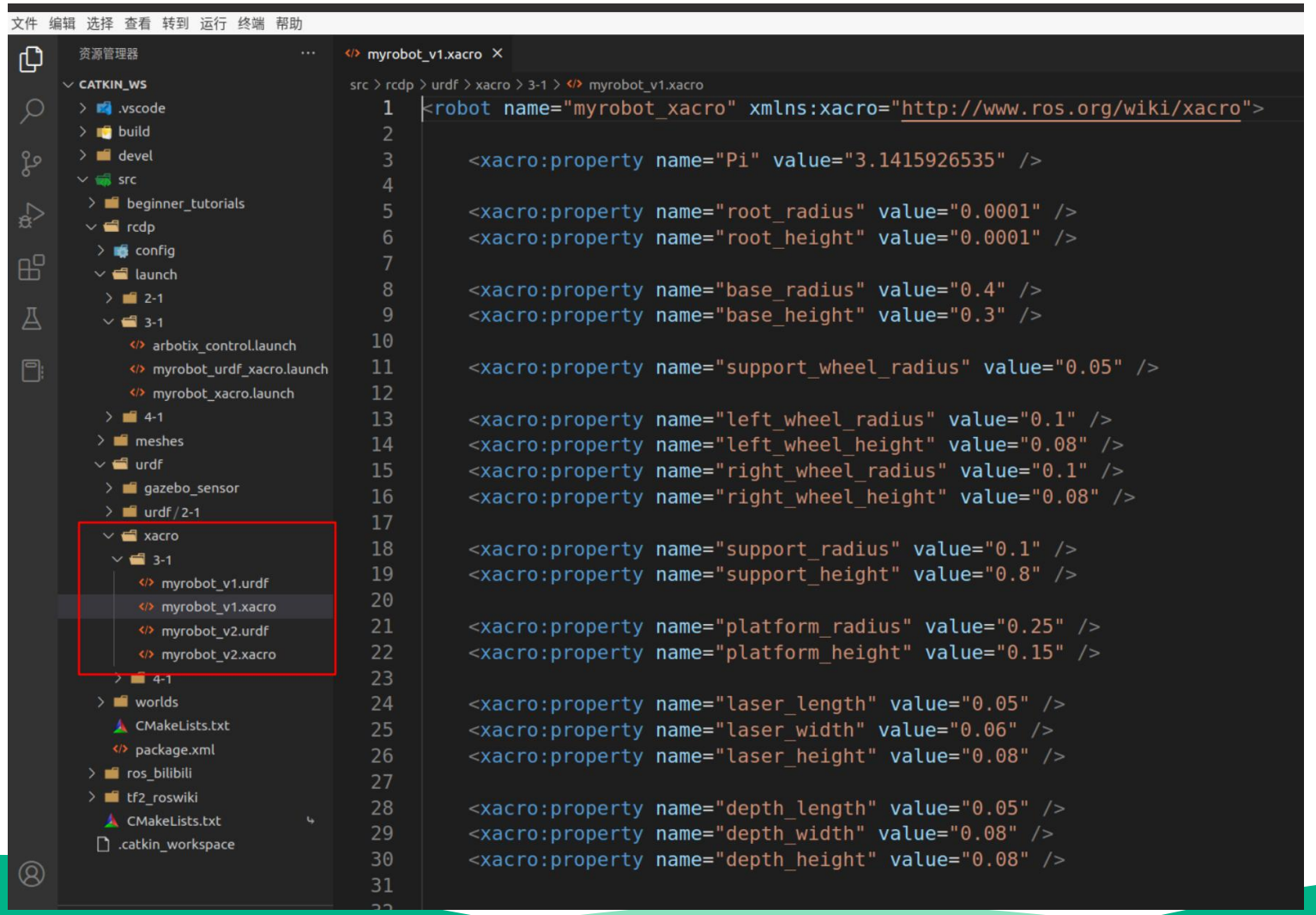
3

Xacro Practice

3.1 Xacro Demo

3.1.1 Xacro案例演示

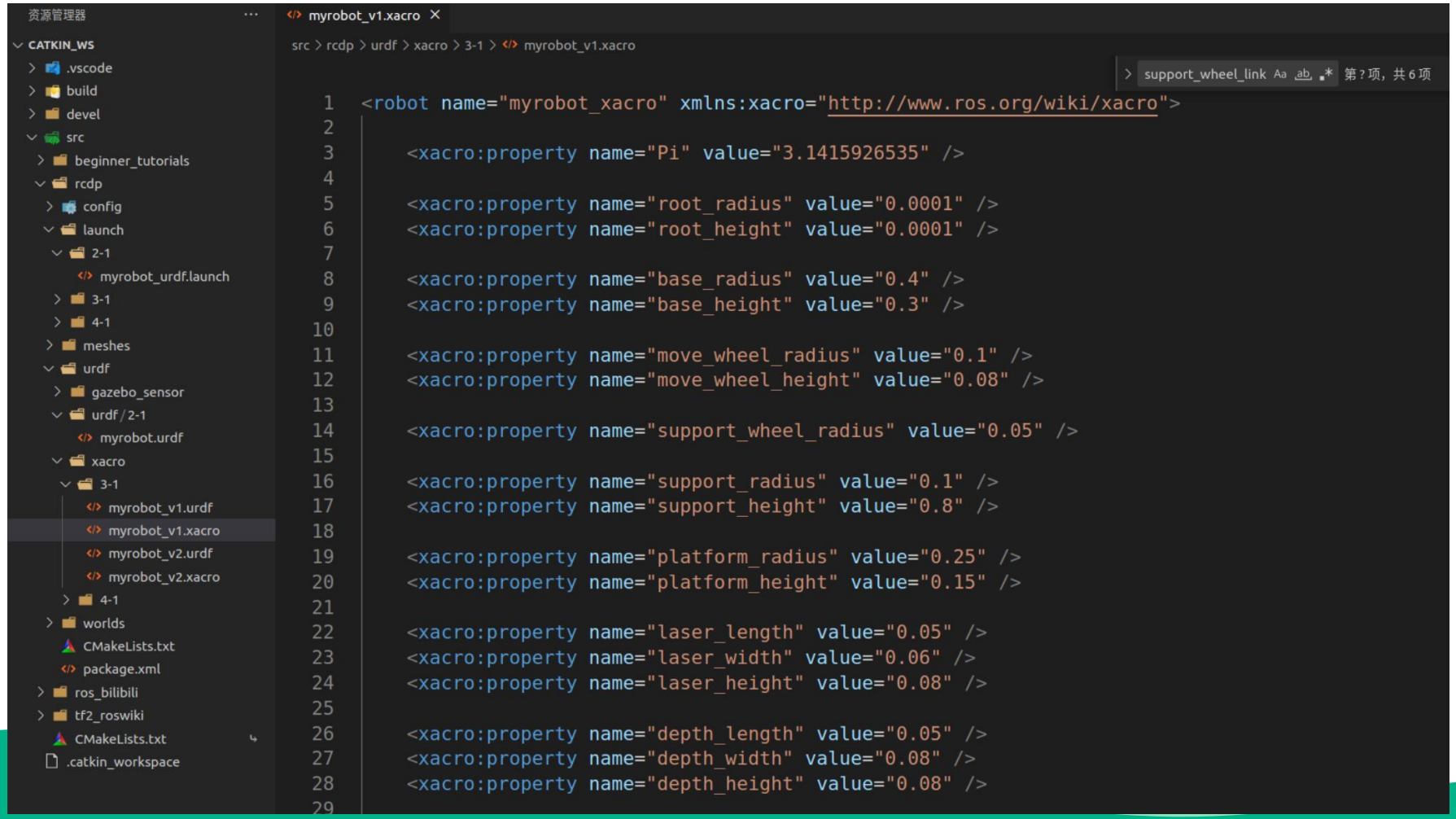
在 urdf 目录的 xacro 目录中，新建 myrobot_v1.xacro 和 myrobot_v2.xacro 文件，通过 xacro 规则编写小车的 xacro 文件



3.1 Xacro Demo

3.1.1 Xacro案例演示

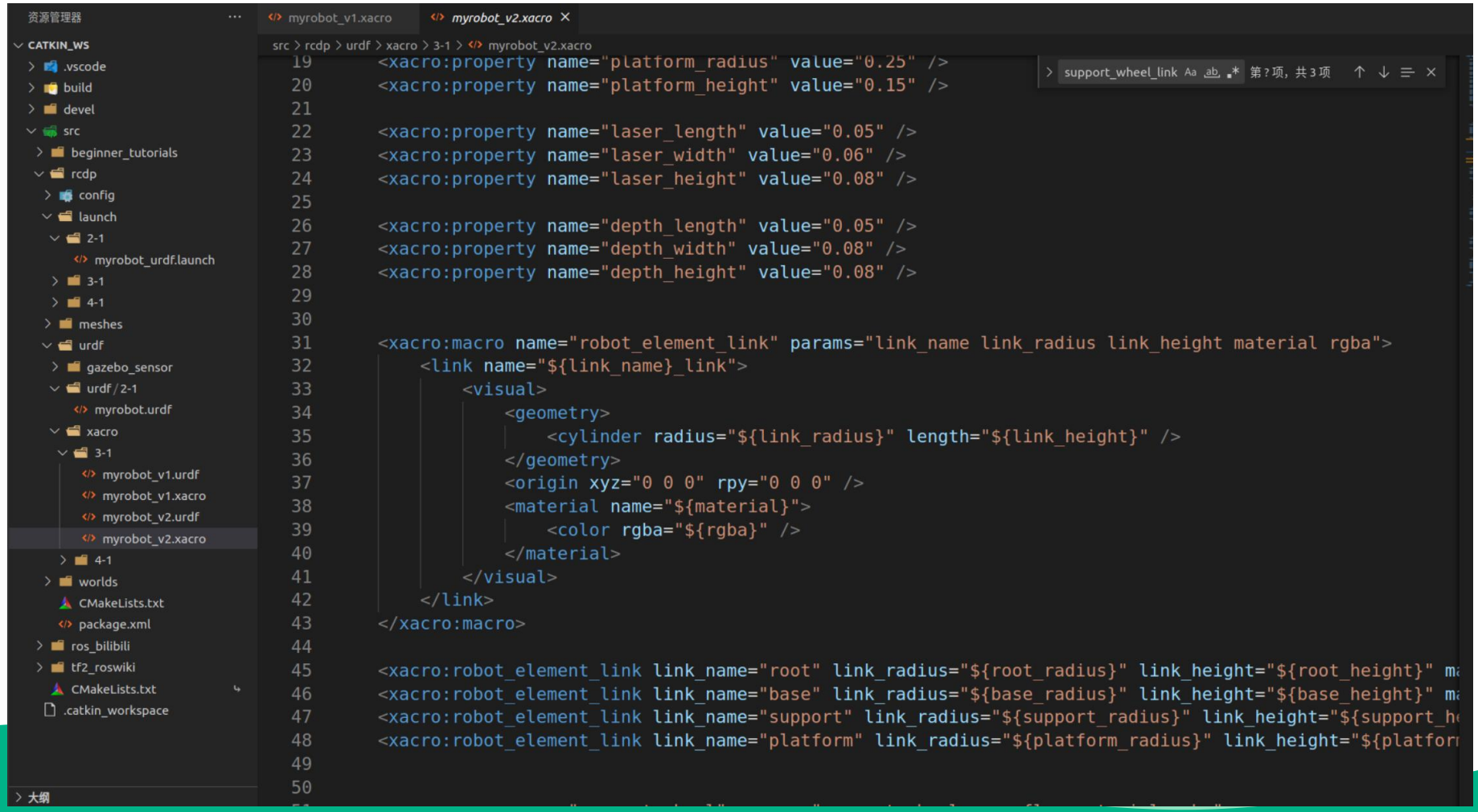
(1) 使用变量的版本-myrobot_v1.xacro



3.1 Xacro Demo

3.1.1 Xacro案例演示

(2) 使用函数的版本-myrobot_v2.xacro



3.1 Xacro Demo

3.1.1 Xacro案例演示

(3) 集成 launch 文件

方式1：先将 Xacro 文件转换出 URDF 文件，然后集成

切换到 xacro 文件所在目录，将 xacro 文件解析成 urdf 文件：

```
rosrun xacro xacro xxx.xacro > xxx.urdf
```

然后再按照之前的集成方式直接整合 launch 文件

3.1 Xacro Demo

3.1.1 Xacro案例演示

(3) 集成launch文件

```
<launch>

    <param name="robot_description" textfile="$(find rcdp)/urdf/xacro/myrobot_v2.urdf" />

    <node pkg="rviz" type="rviz" name="rviz" args="-d $(find rcdp)/config/myrobot.rviz"/>

    <node pkg="joint_state_publisher" type="joint_state_publisher" name="joint_state_publisher" />

    <node pkg="robot_state_publisher" type="robot_state_publisher" name="robot_state_publisher" />

</launch>
```

3.1 Xacro Demo

3.1.1 Xacro案例演示

(3) 集成launch文件

方式2：在 launch 文件中直接加载 Xacro(建议使用)

```
<launch>

    <param name="robot_description" command="$(find xacro)/xacro $(find
rcdp)/urdf/xacro/myrobot_v2.xacro" />

    <node pkg="rviz" type="rviz" name="rviz" args="-d $(find rcdp)/config/myrobot.rviz" />

    <node pkg="joint_state_publisher" type="joint_state_publisher" name="joint_state_publisher" />

    <node pkg="robot_state_publisher" type="robot_state_publisher" name="robot_state_publisher" />

</launch>
```



4

Arbotix Controller

4.1 Arbotix

4.1.1 Overview：http://wiki.ros.org/arbotix

- Arbotix 是一款控制电机、舵机的控制板，并提供相应的 ros 功能包
- Arbotix 可以驱动真实的 Arbotix 控制板，它还提供一个差速控制器，通过接受速度控制指令更新机器人的 joint 状态，从而帮助我们实现机器人在 Rviz 中的运动
- 这个差速控制器在 arbotix_python 程序包中，完整的 arbotix 程序包还包括多种控制器，分别对应 dynamixel 电机、多关节机械臂以及不同形状的夹持器

4.1 Arbotix

4.1.1 Overview

- 实现流程：

1. 安装 Arbotix
2. 创建新功能包(或继续使用rcdp功能包)，准备机器人 URDF、Xacro 文件
3. 添加 Arbotix 配置文件
4. 编写 launch 文件配置 Arbotix
5. 启动 launch 文件并控制机器人模型运动

4.1 Arbotix

4.1.1 Overview

(1) 安装 Arbotix：

1. 命令行安装

```
$ sudo apt install ros-noetic-arbotix
```

(2) 创建新功能包，准备机器人 URDF、Xacro文件

4.1.1 Overview

(3) 添加 arbotix 所需的配置文件：

```
controllers: {
  base_controller: {
    type: diff_controller,
    base_frame_id: root_link,
    base_width: 0.8,
    ticks_meter: 2000,
    Kp: 12,
    Kd: 12,
    Ki: 0,
    Ko: 50,
    accel_limit: 1.0
  }
}
```

urdf/xacro文件中所有link的父节点

4.1 Arbotix

4.1.1 Overview

(4) launch 文件中配置 arbotix 节点：

```
<node name="arbotix" pkg="arbotix_python" type="arbotix_driver" output="screen">
    <rosparam file="$(find rcdp)/config/arbotix_control.yaml" command="load" />
    <param name="sim" value="true" />
</node>
```

➢ 代码讲解：

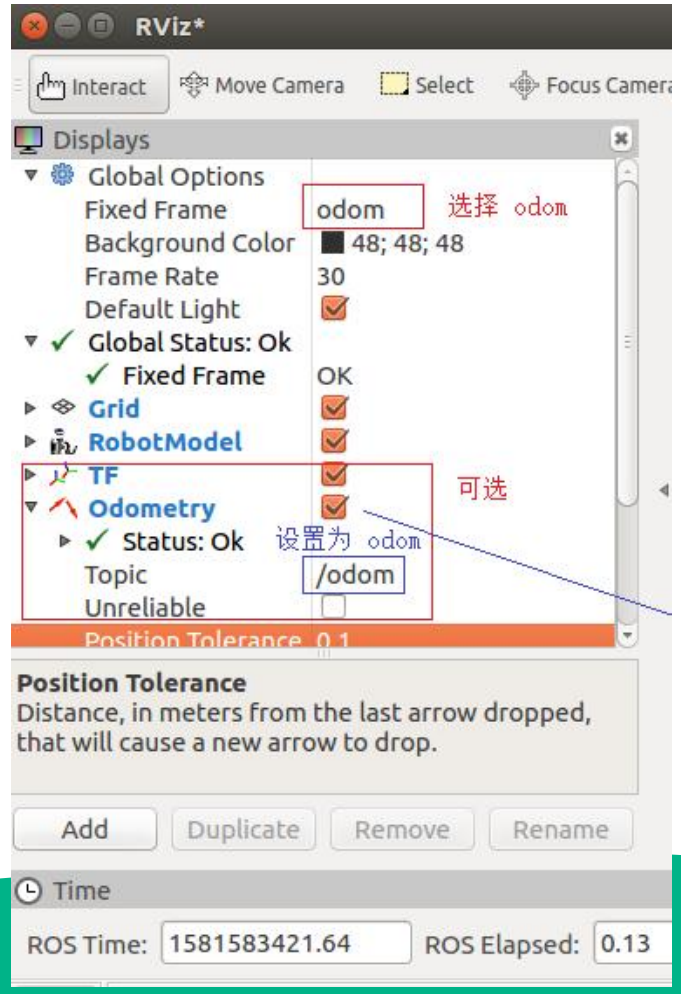
<node> 调用了 arbotix_python 功能包下的 arbotix_driver 节点

<rosparam> arbotix 驱动机器人运行时，需要获取机器人信息，可以通过 file 加载配置文件

<param> 在仿真环境下，需要配置 sim 为 true

4.1.1 Overview

(5) 启动 launch 文件并控制机器人模型运动：




```
ubuntu@Turtlebot3-virtual-machine: ~
ubuntu@Turtlebot3-virtual-machine:~$ rostopic list
/clicked_point
/cmd_vel
/diagnostics
/initialpose
/joint_states
/move_base_simple/goal
/odom
/rosout
/rosout_agg
/tf
/tf_static
```

```
$ rostopic pub /cmd_vel
geometry_msgs/Twist "[0.1,0,0]" "[0,0,0]"
```

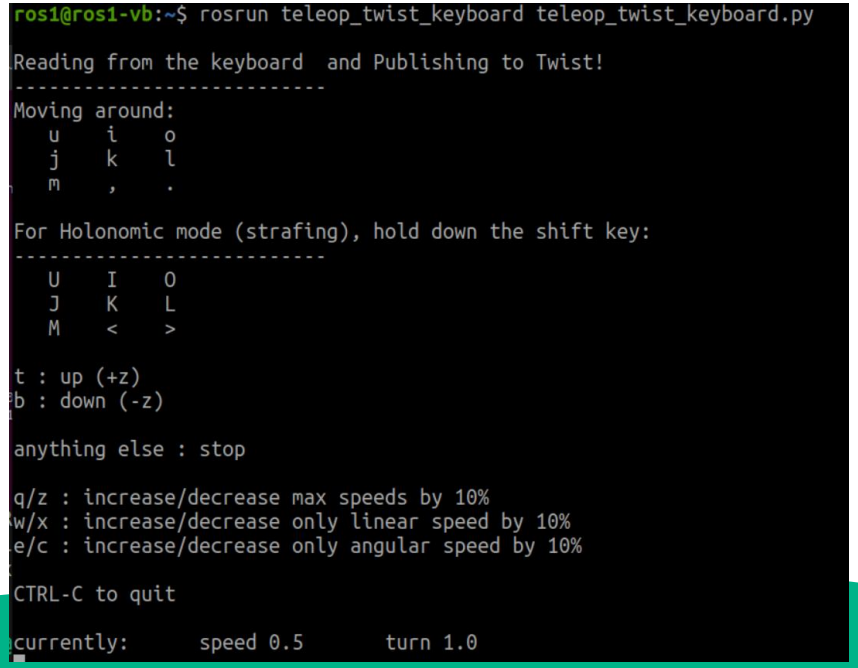
4.1 Arbotix

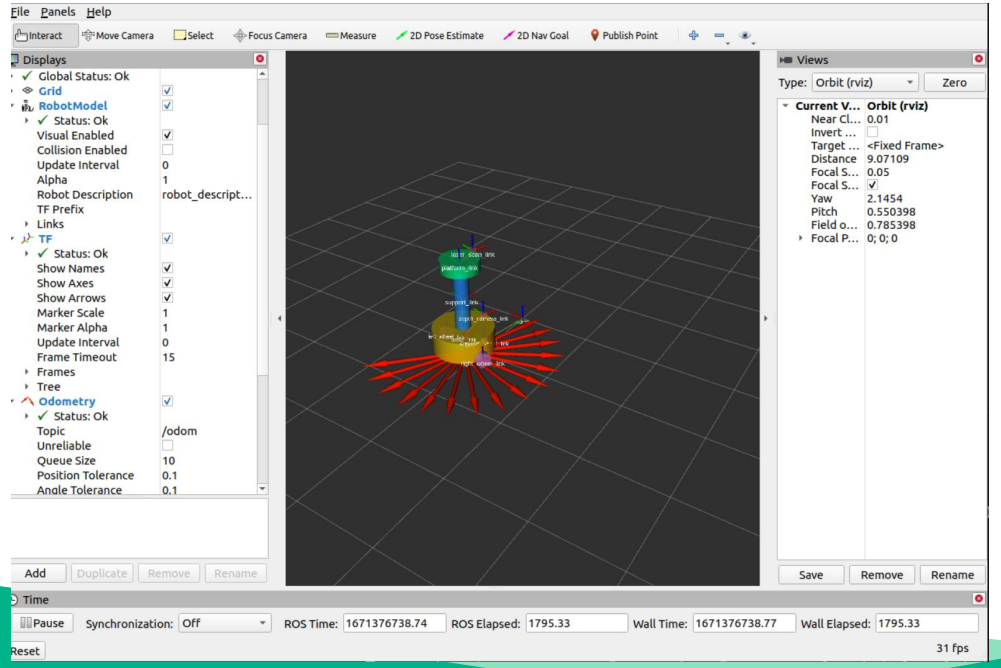
4.1.1 Overview

(6) 通过键盘控制机器人模型运动：

```
$ sudo apt install ros-noetic-teleop-twist-keyboard
```

```
$ rosrun teleop_twist_keyboard teleop_twist_keyboard.py
```





Summary

1. Xacro概述
    a) URDF基本语法
    b) URDF案例演示
    c) Xacro概述
2. Xacro基本语法
    a) 属性与算数运算
    b) 宏
    c) 文件包含
3. Xacro案例演示
    a) 变量版本（v1）
    b) 函数版本（v2）
4. Arbotix 控制器
    a) 使用方法
    b) 键盘控制功能包

Q&A

今日事今日毕

Question?